

Reading and processing geographical raster data in Stata

Kerui Du

School of Management
Xiamen University
Xiamen, China

kerrydu@xmu.edu.cn

Chunxia Chen

School of Management
Xiamen University
Xiamen, China

chenchunxia@stu.xmu.edu.cn

Yang Song

School of Economics
Hefei University of Technology
Hefei, China

ss0706082021@163.com

Shuo Hu

School of Economics
Southwestern University of Finance and Economics
Chengdu, China

advancehs@163.com

Ruipeng Tan

School of Economics
Hefei University of Technology
Hefei, China
tanruipeng@hfut.edu.cn

Abstract. We integrate the Java GeoTools and NetCDF libraries into Stata and introduce a new suite of commands for reading and processing geospatial raster data entirely within Stata. The commands enable Stata users to seamlessly extract data from GeoTIFF and NetCDF files, reproject geographic coordinates, match raster data with geographical locations, and compute zonal statistics. By eliminating round-trips to external GIS or scripting environments, the tools streamline workflows and improve reproducibility of spatial analyses conducted in Stata. We document the command syntax and provide worked examples that illustrate typical use cases—metadata inspection, subsetting, Coordinate Reference Systems harmonization, polygon-based aggregation, and point-based exposure estimation—demonstrating how the package supports end-to-end spatial data preparation and analysis.

Keywords: GeoTIFF, NetCDF, Java, Raster, Geospatial

1 Introduction

Geospatial data has become indispensable across disciplines such as environmental science, economics, and public health. Contemporary research increasingly combines spatial information (e.g., raster and shapefile data) with statistical modeling to address complex empirical questions. Several studies illustrate this trend. For example, Burke et al. (2015) used satellite remote sensing to construct a global agricultural climate risk index, quantifying the nonlinear effects of extreme temperature on crop yields and informing climate adaptation policies. Henderson et al. (2012) developed a statistical framework that incorporated satellite-derived night lights to complement official mea-

asures of income growth. He et al. (2020) employed satellite data to identify agricultural straw burning in China and linked it with air quality and mortality records, providing evidence on the effectiveness of straw recycling policies. More recently, Barwick et al. (2024) addressed the limited availability of pollution data by retrieving ambient air quality indicators from aerosol optical depth (AOD) using the MODIS algorithm, thereby evaluating the costs and benefits of China’s national pollution monitoring and disclosure program.

Although Stata is widely recognized for its strengths in data management and statistical analysis, its functionality for geospatial data processing is highly constrained. In particular, Stata cannot directly import widely used raster formats such as GeoTIFF and NetCDF. As a result, researchers must rely on external software (e.g., ArcGIS, QGIS, R, or Python) to preprocess spatial data before importing the outputs into Stata. This fragmented workflow increases analytical complexity, hinders reproducibility, and ultimately limits the scope of geospatial applications that can be conducted within Stata.

This paper addresses the above limitations by leveraging the Java integration functionality introduced in Stata 19.¹ Building on this feature, we incorporate the GeoTools and NetCDF Java libraries into Stata and develop a suite of commands for reading and processing raster data.² Our implementation focuses on two widely adopted geospatial formats—GeoTIFF and NetCDF—while acknowledging that many other formats (e.g., ASCII, RINEX, OGC KML) are also used in geospatial research. Section 2 provides background on these formats and describes the technical design underlying our package. For broader standards in geospatial data, readers may refer to the Earth Science Data Systems (ESDS) documentation (<https://www.earthdata.nasa.gov/about/standards>).

We introduce seven new Stata commands: `gtiffdisp`, `gtiffread`, `ncdisp`, `ncread`, `crsconvert`, `zonalstats` and `matchgeop`. Together, these commands allow users to directly import, inspect, transform, and analyze GeoTIFF and NetCDF raster files within Stata. Subsequent sections detail the syntax of each command and illustrate their application through empirical examples drawn from real-world research contexts.

2 A brief introduction to GeoTIFF and NetCDF

2.1 The GeoTIFF Format

GeoTIFF is a widely used geospatial raster format that extends the standard Tagged Image File Format (TIFF) by embedding georeferencing metadata, such as the coordinate reference system (CRS), spatial resolution, etc. It encodes spatial information as a grid of pixels and supports multiple data bands, typically stored in band-sequential (BSQ) order.

-
1. The Java integration offers a JShell-like environment where developers can run Java code directly within Stata.
 2. GeoTools is an open-source Java library for geospatial data processing, providing tools to manipulate, analyze, and visualize spatial datasets. NetCDF is an open source toolkit for scientific data processing, enabling cross-platform reading, writing, and analysis of NetCDF/HDF5 datasets

The GeoTIFF format has become the de facto industry standard for GIS and satellite remote sensing applications, with broad support across both open-source and commercial geospatial software platforms. It is particularly well suited for representing two- or three-dimensional (2D or 3D) raster datasets, such as high-resolution satellite imagery and digital elevation models (DEMs).

However, the GeoTIFF format is inherently image-centric and less suited to complex multidimensional scientific datasets such as time series or vertical profiles. Its structure is largely restricted to 2D layers per band, which makes it inefficient for representing sparse data. In meteorological and oceanographic applications—particularly for narrow swath observations from orbiting satellites—GeoTIFF files often contain large amounts of redundant or missing values, leading to wasted storage. Moreover, because these data are commonly stored in floating-point form, high-resolution and multiband GeoTIFF files can become extremely large, imposing substantial demands on both storage and computation.

2.2 The NetCDF Format

NetCDF (Network Common Data Form) is a platform-independent, array-oriented format that structures data using dimensions, variables (multidimensional arrays), and attributes. Unlike GeoTIFF, NetCDF is particularly effective for storing and managing complex multidimensional scientific datasets, such as climate model outputs and long-term time series. Its flexible design accommodates multiple dimensions (e.g., spatial, temporal, vertical) and supports extensive metadata, enabling consistent documentation of data provenance and structure. In addition, NetCDF enjoys broad interoperability, with support across scientific programming environments and GIS software, including Python (e.g., `netCDF4`, `xarray`), MATLAB, QGIS and ArcGIS.

Nevertheless, several limitations remain. First, data values and spatial referencing information are typically stored separately, which complicates real-time computation and data sharing. Second, organizational conventions may differ across data providers, requiring researchers to carefully examine and interpret file structures before analysis. Third, the values stored in NetCDF files are sometimes derived products rather than raw observations, necessitating additional processing when preparing data for visualization or statistical analysis.

3 The workflow of the package

The seven newly developed Stata commands together establish a comprehensive, three-stage workflow for processing GeoTIFF and NetCDF raster data entirely within Stata's statistical environment. As shown in Figure 1, the workflow enables users to progress from raw geospatial files to integrated, analysis-ready datasets suitable for empirical research. A central objective is the quantification of environmental or geographic exposure for areas of interest (AOIs). This is achieved by overlaying a zone layer (e.g., administrative boundaries or grid cells) on a value layer (e.g., nighttime light intensity,

temperature, or pollution). The core operation extracts raster cell values within each zone and computes the required statistics (e.g., mean, sum, minimum or maximum), thereby generating exposure measures at the desired spatial scale.

The workflow proceeds in three stages. The first stage is metadata inspection, in which the commands `gtiffdisp` and `ncdisp` display key information from GeoTIFF and NetCDF files, respectively, including spatial dimensions, coordinate reference systems, and variable attributes. The second stage is data import, where `gtiffread` and `ncread` load raster values into Stata's memory in a vectorized form. The third stage is spatial operations, which enable advanced geospatial analysis. The command `zonalstats` computes summary statistics by aggregating raster cell values within polygonal zones. It operates directly on the input files, thereby avoiding unnecessary vectorization, and incorporates internal logic to address spatial misalignment between zone layers and raster data. For analyses that require localized exposure estimates at discrete point locations (e.g., monitoring stations), the command `matchgeop` can be applied. To ensure consistency, data imported from either format can be pre-processed with `crsconvert`, which harmonizes coordinate reference systems prior to point–raster matching. Together, these steps allow for advanced exposure estimation methods, such as computing Inverse Distance Weighted (IDW) averages of nearby raster cells to represent localized conditions.

4 Java Environment and Dependencies Configuration

The Stata commands developed herein depend on the Java Development Kit (JDK) and external Java libraries. While Stata typically includes a bundled Java environment, the minimum required JDK version for these commands is 17. For Stata 17, users need to manually install JDK 17 or above and set it as the `JAVA_HOME` (see `help java set`). For Stata 18 and above, these versions come pre-bundled with JDK 17 or later, so no additional JDK installation is needed. Additionally, before using the package, users must configure the required Java dependencies. Configuration is needed only once, unless the library files are subsequently moved or deleted.

We provide two distinct setup methods. The first method, which uses precompiled JARs, is the recommended approach as it is compatible with both JDK 17 and JDK 21. Users can configure the environment by running `geotools_init`, compiled for GeoTools-based commands and `netcdf_init`, compiled for NetCDF commands.

The second method, which uses JShell to compile from source code, is only compatible with JDK 17. For GeoTools setup, users can run `geotools_init`, `download plus(geotools)` to automatically download and install the required files. However, the process may take considerable time due to Stata's relatively slow handling of large downloads. A faster alternative is to manually download the GeoTools 34.0 package directly from Sourceforge, unzip it, and initialize the environment with the command: `geotools_init path.to.geotools-34.0/lib, plus(geotools)`, replacing the

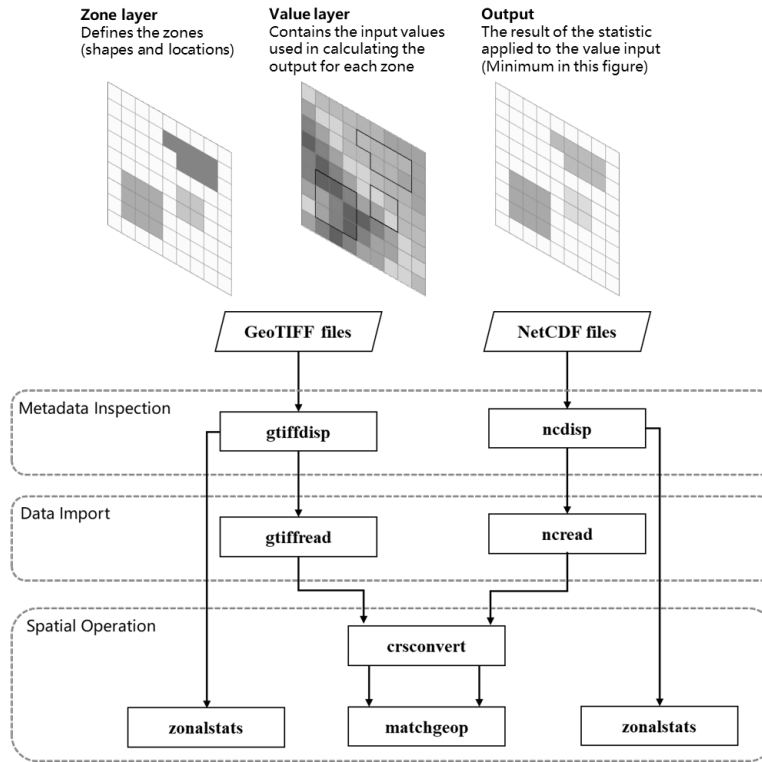


Figure 1: The workflow for the developed command set

path with the actual directory.³ After doing so, configuring the environment by running: `geotools_init path_to_geotools-34.0/lib, plus(geotools)`. For NetCDF setup, users can configure the environment by running: `netcdf_init, download plus(netcdf)`. This command automatically retrieves the required `netcdfAll-5.9.1.jar` file and places it in the appropriate directory within Stata's `sysdir PLUS`.

To further streamline the process, we provide a setup script (`setup.do`) that automates configuration for both Java environments. Detailed instructions are also available in our GitHub repository.⁴

3. <https://sourceforge.net/projects/geotools/files/GeoTools%2034%20Releases/34.0/>

4. <https://github.com/kerrydu/readraster/blob/develop/javaenvconfig.md>

5 Metadata Inspection Commands

5.1 The `gtiffdisp` command

Syntax

`gtiffdisp filename`

filename specifies the GeoTIFF file to be read.

Store results

`gtiffdisp` stores the following in `r()`:

`scalar`

`r(nband)` returns the number of bands.

`r(ncol)` returns the number of columns (width).

`r(nrow)` returns the number of rows (height).

`r(minX)` returns the minimum X coordinate.

`r(minY)` returns the minimum Y coordinate.

`r(maxX)` returns the maximum X coordinate.

`r(maxY)` returns the maximum Y coordinate.

`r(Xcellsize)` returns the X resolution (pixel size).

`r(Ycellsize)` returns the Y resolution (pixel size).

Description

The `gtiffdisp` command displays essential information from a GeoTIFF file, including its dimensions, number of bands, spatial resolution, coordinate reference system and other metadata.

5.2 The `ncdisp` command

Syntax

To view metadata for an entire NetCDF file:

`ncdisp using path_to_ncfile`

To view metadata for a specific variable in a NetCDF file:

`ncdisp varname using path_to_ncfile`

varname specifies variable in the NetCDF file.

path_to_ncfile specifies the NetCDF file to be read.

Store results

`ncdisp` stores the following in `r()`:

`local`

`r(varname)` returns the name of the variable.

`r(dimensions)` returns the dimensions of the variable.

`r(coordinates)` returns the coordinate axes of the variable.

`r(datatype)` returns the datatype of the variable.

Description

The `ncdisp` is used to display comprehensive metadata for NetCDF files. When no variable is specified, it reports global attributes such as the data source and coordinate system. When a variable name is provided, it returns variable-specific details, including associated dimensions, coordinate axes and data type.

6 Data Import Commands

6.1 The `gtiffread` command

Syntax

```
gtiffread filename [ , clear band(#) origin(numlist) size(numlist)
                    crscode(string) ]
```

filename specifies the GeoTIFF file to be read.

Options

`clear` specifies clearing the current dataset to import data.

`band(#)` specifies which band to read from the GeoTIFF, Default is 1.

`origin(numlist)` specifies the starting position for reading ((row col)). It must be used together with the size option. `origin(2 3)` begin the reading process from the cell located at the intersection of the second row and the third column. By default,

it starts from the beginning (1,1).

size(*numlist*) specifies the number of rows and columns to be read. A value of -1 indicates that data should be read until the end of the corresponding dimension. For example, **size**(3 -1) reads three rows and all columns from the specified origin. If not specified, the command reads the entire raster from the origin to the end.

crscode(*string*) specifies the target coordinate reference system (CRS) code. It can be:

- "EPSG:code" (e.g., "EPSG:4326" for WGS84)
- Path to a .tif file with the desired CRS
- Path to a .shp file with the desired CRS

If not specified, it uses the GeoTIFF's native CRS.

Description

The **gtiffread** command is used to extract pixel values with their corresponding coordinates. The command supports reading specific bands, subsetting by origin and size, and optional coordinate system conversion.

6.2 The ncread command

Syntax

```
ncread [varname] using path_to_ncfile, [ clear csv(filename [,replace])
      origin(numlist) size(numlist) ]
```

varname indicates reading the data of the specified *varname* in the NetCDF file. If not specified, the command would display the metadata in the nc file.

path_to_ncfile specifies the NetCDF file to be read.

Options

clear specifies clearing the current dataset.

csv(*filename* [,*replace*]) specifies reading the data from the NetCDF file into the specified CSV file. If *replace* is specified, the existing CSV file will be replaced.

origin(*numlist*) specifies the starting indices of each dimension for reading a section of raster data from the NetCDF file. **origin**(3 2 1) means beginning with the third element in the first dimension, the second element in the second dimension, and the first element in the third dimension.

size(*numlist*) determines the reading count for each dimension. A value of -1 indicates

that the data should be read until the end of the corresponding dimension. `size(1 3 -1)` means reading one and three elements in Dimensions 1 and 2, respectively, and reading the elements in Dimension 3 from the starting index until the end.

Description

The `ncread` command is used to read data for a specified variable from a NetCDF file into Stata. It can read the entire variable or a specified section of data. If varname is not specified, `ncread` displays the meta-information of the NetCDF.

7 Spatial Operation Commands

7.1 The `zonalstats` command

Syntax

For GeoTiff files:

```
zonalstats rasterfilename using shpfile, [ stats(string) band(#) crs(string)
clear]
```

For NetCDF files:

```
zonalstats rasterfilename using shpfile var string, [ stats(string)
crs(string) origin(numlist) size(numlist) clear]
```

rasterfilename can be a GeoTIFF file (*.tif* or *.tiff*) or NetCDF file (*.nc*). The command automatically detects the file type and uses the appropriate processing method.

shpfile specifies a shpfile to read. It requires a complete shapefile set (*.shp*, *.shx*, *.dbf* and *.prj* files) in the same folder.

Options

`clear` specifies clearing the current data set to import data.

`stats(string)` specifies which statistics to calculate. Default is *avg* if not specified.

Valid options are:

count: the number of pixels in the zone

avg: the average pixel value

min: the minimum pixel value

max: the maximum pixel value

std: the standard deviation of pixel values

sum: the sum of pixel values

crs(*string*) specifies coordinate reference system for the raster data(e.g., “EPSG:4326”). If the raster file contains CRS information, this option is ignored.

band(*#*) specifies which raster band to use in multi-band raster files. Default is 1 (first band). This option is only applicable when performing zonal statistics on GeoTIFF files.

var(*string*) specifies variable name in the NetCDF file to process. It is required for NetCDF files

origin(*numlist*) specifies the starting indices of each dimension for rslicing the NetCDF variable. This option is only applicable when performing zonal statistics on NetCDF files.

size(*numlist*) specifies the number of elements to read along each dimension when slicing the NetCDF variable. Only up to two dimensions can have a size greater than 1. The requirement that the data must be a 2D grid is enforced because zonal statistics operations are defined for spatial data distributed over two dimensions: typically latitude and longitude (or x and y). This option is only applicable when performing zonal statistics on NetCDF files.

Description

The **zonalstats** command computes zonal statistics for each polygon in a shapefile using the raster cell values that fall within its boundaries. It supports automatic coordinate system transformation when the shapefile and raster use different projections, and optimizes performance by restricting raster processing to the area intersecting the shapefile’s extent.

7.2 The crsconvert command

Syntax

```
crsconvert varlist [ , gen(string) from(string) to(string) ]
```

varlist must contain exactly two numeric variables representing x and y coordinates in the source coordinate reference system.

Options

gen(*prefix*) specifies the prefix for the two new variables that will contain the transformed coordinates. The new variables will be named *prefix.x* and *prefix.y*.

`from(string)` specifies the source coordinate reference system. It might be provided in EPSG format (e.g., “EPSG:4326”). The coordinate reference system can be provided in EPSG format (e.g., “EPSG:4326”). Alternatively, users can specify a GeoTIFF (.tif/.tiff), Shapefile (.shp), or NetCDF (.nc) file to automatically extract the coordinate reference system from the file. This is particularly useful when the coordinate system does not correspond to a standard EPSG code but includes custom projection parameters. If a file is provided, the command will automatically extract the CRS information from it.

`to(string)` specifies the source coordinate reference system. It might be provided in EPSG format (e.g., “EPSG:4326”). Alternatively, users can specify a GeoTIFF (.tif/.tiff), Shapefile (.shp), or NetCDF (.nc) file to denote the source of the coordinate reference system.

Description

The `crsconvert` command converts coordinates from one coordinate reference system to another. It creates two new variables storing the transformed coordinates.

7.3 The matchgeop command

Syntax

```
matchgeop baseid baselat baselon using nborfile, neighbors(nborid nborlat
    nborlon) [ uframe within(#) mile gen(newvar) nearcount(#)
    bearing(newvar) nsplit(#) ]
```

Options

`neighbors(varlist)` specifies the neighbor variables to consider for matching. Must contain 3 variables in the order: id, latitude and longitude.

`uframe` specifies the using data is in a frame. By default, the using data is Stata data format in the disk.

`within(#)` specifies the range within which neighbors are considered.

`mile` specifies that distances should be measured in miles. Default is kilometers.

`nearcount(#)` specifies the number of nearest neighbors to consider. Must be a positive number.

`gen(newvar)` specifies the name of the new variable to store the distance.

`bearing(newvar)` specifies the name of the new variable to store the bearing.

`nsplit(#)` specifies the number of splits for looping. `nsplit = 1` does one loop with

all observations (requires large memory), `nsplit = 2` does two loops with half of the observations, and so on.

Description

`matchgeop` merges datasets based on a specified variable list and neighbors. It finds the nearest neighbors based on geographic location and generates a linked dataset.

8 Examples

8.1 GeoTIFF Data Processing

Display the Metadata of the GeoTIFF File

The metadata of a GeoTIFF file can be inspected using the `gtiffdisp` command, which reports information on bands, spatial characteristics, coordinate system, units, and other metadata fields. As an illustration, we use the file `DMSP-like2020.tif`, which provides improved DMSP-OLS-like nighttime light data for China by integrating observations from the Defense Meteorological Satellite Program-Operational Linescan System (DMSP-OLS) and the Suomi National Polar-orbiting Partnership-Visible Infrared Imaging Radiometer Suite (SNPP-VIIRS)(Wu 2021).

```
. gtiffdisp DMSP-like2020.tif
=== Band Information ===
Number of bands: 1
Band 1 : GRAY_INDEX          | NoData: Not defined
=== Spatial Characteristics ===
X range: [-2643772.8565 ~ 2212227.1435]
Y range: [1871896.5263 ~ 5926896.5263]
Resolution: X=1000.0000 units/pixel, Y=1000.0000 units/pixel
=== Coordinate System ===
CRS Name: PCS Name = WGS_1984_Albers
CRS WKT: PROJCS["PCS Name = WGS_1984_Albers",
  GEOGCS["WGS 84",
    DATUM["World Geodetic System 1984",
      SPHEROID["WGS 84", 6378137.0, 298.257223563, AUTHORITY["EPSG","7030"]],
      AUTHORITY["EPSG","6326"]],
    PRIMEM["Greenwich", 0.0, AUTHORITY["EPSG","8901"]],
    UNIT["degree", 0.017453292519943295],
    AXIS["Geodetic longitude", EAST],
    AXIS["Geodetic latitude", NORTH],
    AUTHORITY["EPSG","4326"]],
  PROJECTION["Albers_Conic_Equal_Area"],
  PARAMETER["central_meridian", 105.0],
  PARAMETER["latitude_of_origin", 0.0],
  PARAMETER["standard_parallel_1", 25.0],
  PARAMETER["false_easting", 0.0],
  PARAMETER["false_northing", 0.0],
  PARAMETER["standard_parallel_2", 47.0],
  UNIT["m", 1.0],
  AXIS["Easting", EAST],
```

```

    AXIS["Northing", NORTH]]
=== Units ===
X unit: m
Y unit: m
=== Filtered Metadata ===
ImageN.ImageReader      : it.geosolutions.imageioimpl.plugins.tiff.TIFFIm...
image_height            : 4055
image_min_x_coord       : 0
image_width             : 4856
ImageN.ImageReadParam    : it.geosolutions.imageio.plugins.tiff.TIFFImageR...
image_min_y_coord       : 0

```

The output provides detailed information for the GeoTIFF file DMSP-like2020.tif. First, it contains a single raster band named “GRAY_INDEX”, for which no nodata value is defined. Second, the CRS is identified as “WGS_1984_Albers”, which applies the Albers Equal Area Conic projection. The Well-Known Text (WKT) representation specifies the projection parameters in detail. Third, the spatial characteristics indicate that the dataset spans an X range from approximately -2,643,772.86 to 2,212,227.14 meters and a Y range from 1,871,896.53 to 5,926,896.53 meters. The spatial resolution is 1,000 meters per pixel in both X and Y directions, providing a consistent and detailed representation of the study area.

Read the GeoTIFF file for a specific region

The following example demonstrates how to extract nighttime light data for the year 2020 in Hunan Province from the GeoTIFF file DMSP-like2020.tif using the `gtiffread` command. Hunan is selected as an illustrative case because it is a major Chinese province with diverse socioeconomic and environmental characteristics, while its moderate geographic extent provides a tractable spatial scale that balances data complexity and computational efficiency.

The first step is to determine the coordinate range of Hunan Province. As shown above, the file DMSP-like2020.tif is projected in the “WGS_1984_Albers” coordinate system. To ensure consistency, the shapefile `hunan.shp` must therefore be transformed into the same CRS using the `crsconvert` command. Once converted, the coordinate extent of Hunan Province can be obtained in the “WGS_1984_Albers” system.

To guarantee complete coverage of the province, the boundary is expanded by $\pm 2,000$ meters. Based on this adjusted extent, the starting index and dimensions of the corresponding region in DMSP-like2020.tif are computed. To identify the vertical (y-axis) range of the raster, the first column is read with the option `origin(1 1) size(-1 1)`. Similarly, the horizontal (x-axis) range is determined by reading the first row with `origin(1 1) size(1 -1)`.

```

. spshape2dta hunan.shp, replace
(importing .shp file)
(importing .dbf file)
(creating _ID spatial-unit id)
(creating _CX coordinate)
(creating _CY coordinate)

```

```

file hunan_shp.dta created
file hunan.dta      created
. use "hunan.dta",clear
. crsconvert _CX _CY, gen(alber) from(hunan.shp) to(DMSP-like2020.tif)
Converting coordinates from CRS:
Source CRS: GEOGCS["GCS_WGS_1984",
  DATUM["D_WGS_1984",
    SPHEROID["WGS_1984", 6378137.0, 298.257223563]],
  PRIMEM["Greenwich", 0.0],
  UNIT["degree", 0.017453292519943295],
  AXIS["Longitude", EAST],
  AXIS["Latitude", NORTH]]
Target CRS: PROJCS["PCS Name = WGS_1984_Albers",
  GEOGCS["WGS 84",
    DATUM["World Geodetic System 1984",
      SPHEROID["WGS 84", 6378137.0, 298.257223563, AUTHORITY["EPSG","7030"]],
      AUTHORITY["EPSG","6326"]],
    PRIMEM["Greenwich", 0.0, AUTHORITY["EPSG","8901"]],
    UNIT["degree", 0.017453292519943295],
    AXIS["Geodetic longitude", EAST],
    AXIS["Geodetic latitude", NORTH],
    AUTHORITY["EPSG","4326"]],
  PROJECTION["Albers_Conic_Equal_Area"],
  PARAMETER["central_meridian", 105.0],
  PARAMETER["latitude_of_origin", 0.0],
  PARAMETER["standard_parallel_1", 25.0],
  PARAMETER["false_easting", 0.0],
  PARAMETER["false_northing", 0.0],
  PARAMETER["standard_parallel_2", 47.0],
  UNIT["m", 1.0],
  AXIS["Easting", EAST],
  AXIS["Northing", NORTH]]

.
. qui sum alber_CX
. local maxX = r(max)+2000
. local minX = r(min)-2000
.
. qui sum alber_CY
. local maxY = r(max)+2000
. local minY = r(min)-2000
.
. gtiffread DMSP-like2020.tif, origin(1 1) size(-1 1) clear
. gen n=_n
. sum n if y>`minY` & y<`maxY`

```

Variable	Obs	Mean	Std. dev.	Min	Max
n	400	3006.5	115.6143	2807	3206

```

. local start_row = r(min)
. local n_rows = r(N)
.
. gtiffread DMSP-like2020.tif, origin(1 1) size(1 -1) clear
. gen n=_n
. sum n if x>`minX` & x<`maxX`

```

Variable	Obs	Mean	Std. dev.	Min	Max
n	385	3292	111.2842	3100	3484

```

. local start_col = r(min)
. local n_cols = r(N)
.
. gtiffread DMSP-like2020.tif, origin(`start_row` `start_col`) ///
> size(`n_rows` `n_cols`) clear
.
. save DMSP-like2020.dta, replace
file DMSP-like2020.dta saved

```

Figure 2 illustrates the spatial distribution of nighttime light brightness across Hunan Province in 2020. The `heatplot` command was employed for generating the visualization (Jann 2019). The displayed image reflects a bounding rectangle rather than an exact provincial boundary, owing to the pixel-based slicing mechanism of the `gtiffread` command. This approach preserves the original raster structure and avoids interpolation, thereby maintaining the analytical integrity of the data. The results show a highly uneven spatial distribution of nighttime light, with pronounced clusters of brightness corresponding to major urban centers in the province, such as Changsha.

```

. use DMSP-like2020.dta, clear
. heatplot ///
> value y x, color(Greys, reverse) level(6) ///
> xlabel(, angle(45)) graphr(c(white))

```

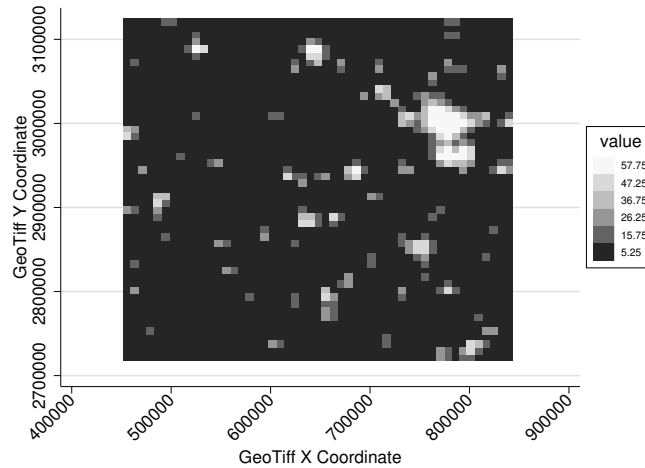


Figure 2: Spatial Distribution of Nighttime Light Brightness

Calculate average nighttime light intensity for cities in Hunan

This part illustrates how to calculate the average nighttime light intensity for Hunan Province from DSMP-like2020 with zonalstats.

```
. zonalstats DMSP-like2020.tif using hunan.shp, stats("avg") clear
GeoTIFF CRS detected: PCS Name = WGS_1984_Albers. User-provided CRS is ignored.
Shapefile CRS: GCS_WGS_1984
Reprojecting shapefile from GCS_WGS_1984 to PCS Name = WGS_1984_Albers
Shapefile bounds for raster reading: ReferencedEnvelope[373271.4002242747 :
> 918728.259838356, 2596264.786161866 : 3202467.2929103803] DefaultProjected
> CRS[PCS Name = WGS_1984_Albers] AXIS["Easting", EAST] AXIS["Northing", NORTH]
Raster envelope: ReferencedEnvelope[-2643772.8565207715 : 2212227.1434792285
> , 1871896.5262559392 : 5926896.526255939] DefaultProjectedCRS[PCS Name= WGS
> _1984_Albers] AXIS["Easting", EAST] AXIS["Northing", NORTH]
Using intersection bounds: ReferencedEnvelope[373271.4002242747 : 918728.2598383
> 56, 2596264.786161866 : 3202467.2929103803] DefaultProjectedCRS[PCS Name= WGS
> _1984_Albers] AXIS["Easting", EAST] AXIS["Northing", NORTH]
Successfully created optimized read parameters
Successfully read raster data with optimization
Total features: 14
Created string variable: z_ShiName (length 16)
Created numeric variable: z_Shape_Leng
Created numeric variable: z_Shape_Area
Created string variable: z_Name (length 16)
Created numeric variable: z_lon
Created numeric variable: z_lat
Created numeric variable: avg
Data successfully exported to Stata dataset.

. list z_Name avg
```

	z_Name	avg
1.	Changde	3.2560185
2.	Chenzhou	2.0607766
3.	Hengyang	3.2285136
4.	Huaihua	1.2616182
5.	Loudi	3.7707743
6.	Shaoyang	1.7991827
7.	Xiangtan	7.2288051
8.	Xiangxi	1.458301
9.	Yiyang	2.4555944
10.	Yongzhou	2.0996052
11.	Yueyang	3.8674934
12.	Zhangjiajie	1.7823899
13.	Changsha	11.616603
14.	Zhuzhou	3.8699787

```
. save "hunan_light.dta", replace
file hunan_light.dta saved
```

The output reveals that the GeoTIFF and shapefile datasets use different projection coordinate systems. To address this inconsistency, the command first reprojects the shapefile to align with the raster data. Subsequently, the raster cells overlapping with the shapefile's extent are extracted, and zonal statistics are computed for each

administrative unit. The visualization is generated with the `geoplot` command (Jann 2023). Figure 3 displays the resulting Average Nighttime Light Index (ANLI) for Hunan Province. The results highlight a pronounced concentration of light intensity in the Chang-Zhu-Tan metropolitan cluster, underscoring its role as the province's economic, cultural and demographic hub.

```
. use hunan.dta, clear
.
. rename Name z_Name
. merge 1:1 z_Name using hunan_light.dta,nogen
(variable z_Name was str11, now str16 to accommodate using data's values)
```

Result	Number of obs
Not matched	0
Matched	14

```
. save hunan_light.dta, replace
file hunan_light.dta saved
.
. geoframe create region ///
>     "hunan_light.dta", id(_ID) centroids(_CX _CY) ///
>     shp(hunan_shp.dta) replace
(creating frame region from hunan_light.dta)
(creating frame region_shp from hunan_shp.dta)
      Frame name: region [make current]
      Frame type: attribute
      Feature type: <none>
      Number of obs: 14
      Unit ID: _ID
      Coordinates: _CX _CY
      Linked shape frame: region_shp
.
. geoplot ///
>     (area region avg, color(Greys, reverse) ///
>         level(6, quantile weight(avg))) ///
>     (line region, lwidth(vthin)), legend(position(sw))
```

Calculate 80km-radius IDW nighttime light for cities in Hunan

While the `zonalstats` command aggregates raster values across polygonal regions defined by shapefiles, the subsequent step employs the `matchgeop` command to calculate the localized exposure using an Inverse Distance Weighting (IDW) approach. This method shifts the analytical focus from broad administrative units to urban cores. The calculation is based on nighttime light data stored in the previously generated `DMSP-like2020.dta` file, combined with the geographic coordinates of city centroids obtained from `hunan_city.dta`.

```
. use "DMSP-like2020.dta", clear
. crsconvert x y, gen(wsg84_) from(DMSP-like2020.tif) to(hunan.shp)
Converting coordinates from CRS:
Source CRS: PROJCS["PCS Name = WGS_1984_Albers",
      GEOGCS["WGS 84",
```

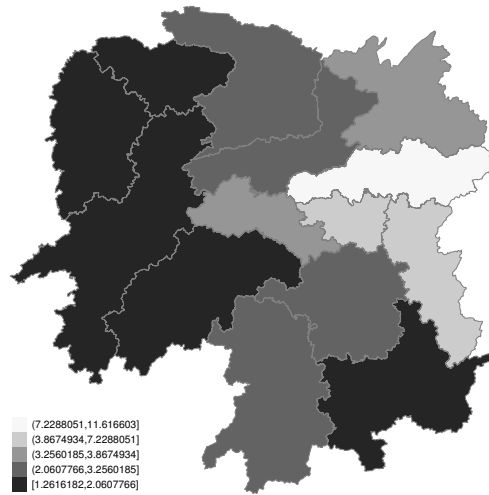


Figure 3: Average nighttime light intensity in Hunan

```
DATUM["World Geodetic System 1984",
      SPHEROID["WGS 84", 6378137.0, 298.257223563, AUTHORITY["EPSG","7030"]],
      AUTHORITY["EPSG","6326"]],
PRIMEM["Greenwich", 0.0, AUTHORITY["EPSG","8901"]],
UNIT["degree", 0.017453292519943295],
AXIS["Geodetic longitude", EAST],
AXIS["Geodetic latitude", NORTH],
AUTHORITY["EPSG","4326"]],
PROJECTION["Albers_Conic_Equal_Area"],
PARAMETER["central_meridian", 105.0],
PARAMETER["latitude_of_origin", 0.0],
PARAMETER["standard_parallel_1", 25.0],
PARAMETER["false_easting", 0.0],
PARAMETER["false_northing", 0.0],
PARAMETER["standard_parallel_2", 47.0],
UNIT["m", 1.0],
AXIS["Easting", EAST],
AXIS["Northing", NORTH]]
Target CRS: GEOGCS["GCS_WGS_1984",
  DATUM["D_WGS_1984",
    SPHEROID["WGS_1984", 6378137.0, 298.257223563]],
  PRIMEM["Greenwich", 0.0],
  UNIT["degree", 0.017453292519943295],
  AXIS["Longitude", EAST],
  AXIS["Latitude", NORTH]]

. gen n=_n
. save "light_china.dta", replace
file light_china.dta saved

.
. use "hunan_city.dta", clear
. matchgeop ORIG_FID lat lon using light_china.dta, neighbors(n wsg84_y wsg84_x)
> within(80) gen(distance)
```

```

.
. merge m:1 n using light_china.dta, keep(3)
      Result                Number of obs
-----
Not matched                    0
Matched                      206,615  (_merge==3)

. drop _merge

.
. drop if value==.
(0 observations deleted)
. gen weight = 1/distance
. gen weighted_light = value * weight
. bysort city: egen sum_weighted_light = total(weighted_light)
. bysort city: egen total_weight = total(weight)
. gen idw_light = sum_weighted_light / total_weight
.
. duplicates drop city, force
Duplicates in terms of city
(206,601 observations deleted)

```

Figure 4 depicts the distribution of nighttime light intensity in 2020 calculated using the IDW approach. Consistent with expectations, Changsha exhibits the highest concentration of light intensity. Notably, the IDW method produces peak values nearly twice as high as those obtained from the zonal average in Figure 3. This discrepancy arises because zonal averaging dilutes urban brightness by incorporating surrounding rural areas, whereas IDW emphasizes localized intensity at city cores.

```

. rename city Name
. merge m:1 Name using hunan.dta
      Result                Number of obs
-----
Not matched                    0
Matched                      14  (_merge==3)

.
. save "hunan_light.dta", replace
file hunan_light.dta saved

.
. geoframe create region ///
>   "hunan_light.dta", id(_ID) centroids(_CX _CY) ///
>   shp(hunan_shp.dta) replace
(creating frame region from hunan_light.dta)
(creating frame region_shp from hunan_shp.dta)
      Frame name: region [make current]
      Frame type: attribute
      Feature type: <none>
      Number of obs: 14
      Unit ID: _ID
      Coordinates: _CX _CY
      Linked shape frame: region_shp
.

```

```
. geoplots ///
> (area region idw_light , color(Greys, reverse) ///
>   level(6, quantile weight(idw_light))) ///
> (line region, lwidth(vthin)), legend(position(sw))
```

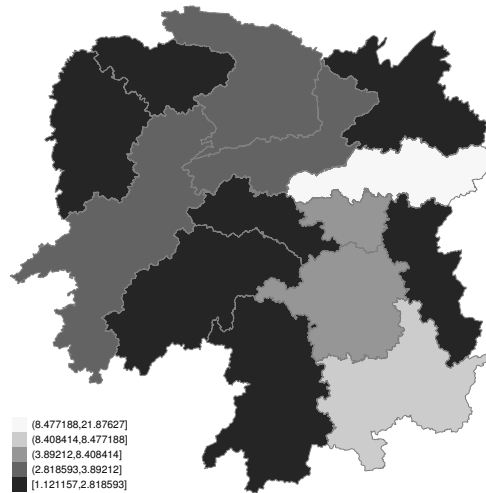


Figure 4: IDW nighttime light intensity at the geocentric center of cities in Hunan

8.2 NetCDF Data Processing

Display the Metadata of the NetCDF File

The `ncdisp` command can be used to inspect the metadata of a NetCDF file. As an example, we illustrate its application using a file from the NEX-GDDP-CMIP6 dataset, which is hosted on Amazon Web Services (AWS).⁵ The NEX-GDDP-CMIP6 dataset contains globally downscaled climate scenarios derived from General Circulation Model (GCM) runs conducted under the Coupled Model Intercomparison Project Phase 6 (CMIP6). These scenarios are developed across two of the four “Tier 1” greenhouse gas emissions pathways, known as Shared Socioeconomic Pathways (SSPs). In this example, we use the file `tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc`. The `ncdisp` output provides comprehensive information about the file, including global attributes, dimensions, and variables.

```
. ncdisp using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc
=== File Structure ===
[Global Attributes]
_NCPProperties          : version=1|netcdfversion=4.4.1.1|hdf5libversi...
```

5. NASA Earth Exchange Global Daily Downscaled Projections (NEX-GDDP-CMIP6) was accessed on October 5, 2025 from <https://registry.opendata.aws/nex-gddp-cmip6>. NEX-GDDP-CMIP6 data was accessed on October 5, 2025 from <https://registry.opendata.aws/nex-gddp-cmip6>.

```

activity          : NEX-GDDP-CMIP6
contact           : Dr. Rama Nemani: rama.nemani@nasa.gov, Dr. Brid...
Conventions       : CF-1.7
creation_date     : 2021-10-04T14:25:12.081609+00:00
frequency         : day
institution       : NASA Earth Exchange, NASA Ames Research Center,...
variant_label     : r1i1p1f1
product          : output
realm             : atmos
source            : BCSD
scenario          : ssp245
references        : BCSD method: Thrasher et al., 2012, Hydrol. Ear...
version           : 1.0
tracking_id       : 6ab2a874-d3fe-484a-a7e8-82df448e364f
title            : BCC-CSM2-MR, r1i1p1f1, ssp245, global downscale...
resolution_id     : 0.25 degree
history           : 2021-10-04T14:25:12.081609+00:00: install globa...
disclaimer        : This data is considered provisional and subject...
external_variables : areacella
cmip6_source_id   : BCC-CSM2-MR
cmip6_institution_id : BCC
cmip6_license     : CC-BY-SA 4.0
_CoordSysBuilder  : ucar.nc2.internal.dataset.conv.CF1Convention

[Dimensions]
Name      Length  Attribute
time      365     Coordinate
lat       600     Coordinate
lon       1440    Coordinate

[Variables]
Name      Dimensions              Type
tas       time lat lon           float
time      time                   double
lat       lat                    double
lon       lon                    double

[Groups]

```

Next, we use `ncdisp` to examine the attributes of the variable *tas* within this file. The results indicate that *tas* represents daily near-surface air temperature, recorded in Kelvin (*K*). This variable has three dimensions—time, lat and lon—with a structure of (365, 400, 1440). Missing values are coded with a value of `1.0E20`.

```

. ncdisp tas using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc
=== Dimensions ===
Dimension      Length  Coordinate
time           365     [Yes]
lat            600     [Yes]
lon            1440    [Yes]
=== Scale/Offset Parameters ===
missing value  : 100000002004087730000.000000 (Type: float)
FillValue      : 100000002004087730000.000000 (Type: float)
=== Attributes ===
missing_value   : 1.0E20
cell_measures   : area: areacella
cell_methods    : area: mean time: maximum (interval: 5 minutes)
original_name    : TREFHTMX
comment         : near-surface (usually, 2 meter) air temperature; derived

```

```

> from downscaled tasmax & tasmin
units           : K
long_name       : Daily Near-Surface Air Temperature
standard_name   : air_temperature
_fillvalue      : 1.0E20
_chunksizes     : 1
=== Metadata ===
Units           : K (original: K)
Shape           : [365, 600, 1440]
Data Type       : float
=== Variable Structure ===
Name: tas                      Type: float

```

We also inspect the metadata of the variable *time*. Its unit is defined as “days since 2040-01-01”, meaning that each value represents the number of days elapsed since January 1, 2040, with fractional values corresponding to portions of a day. This temporal encoding ensures compatibility with standardized climate modeling practices.

```

. ncdisp time using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc
=== Dimensions ===
Dimension      Length  Coordinate
time           365     [Yes]
=== Scale/Offset Parameters ===
=== Attributes ===
units          : days since 2040-01-01
standard_name  : time
long_name      : time
calendar       : 365_day
axis           : T
_chunksizes    : 365
_coordinateAxisType : Time
=== Metadata ===
Units          : days since 2040-01-01 (original: days since 2040-01-01)
Shape          : [365]
Data Type      : double
=== Variable Structure ===
Name: time                      Type: double

```

Read the NetCDF file

The NetCDF file `tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc` is stored in the WGS84 coordinate system, which allows direct slicing and reading based on latitude and longitude ranges.

Hunan Province is located between 24°38'N–30°08'N latitude and 108°47'E–114°15'E longitude. In the following example, we load grid data covering this region, defined by the ranges 108°–115°(longitude) and 24°–31°(latitude). To accomplish this, we first extract the variable *lon* from the NetCDF file into Stata, generate an index variable *n*, and determine both the origin index and the size for the specified range (108°–115°). A similar procedure is applied to the variable *lat*, enabling us to obtain the corresponding indices for the latitude range.

```

. ncread lon using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc, clear
Sucessfully import 1440 Obs into Stata.
. gen n=_n
. qui sum n if lon>=108 & lon<=115
. local lon_start = r(min)
. local lon_count = r(N)
.
. ncread lat using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc, clear
Sucessfully import 600 Obs into Stata.
. gen n=_n
. qui sum n if lat>=24 & lat<=31
. local lat_start = r(min)
. local lat_count = r(N)
.
. ncread tas using tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc, ///
> clear origin(1 `lat_start' `lon_start') ///
> size(-1 `lat_count' `lon_count')

Sucessfully import 286160 Obs into Stata.
.
. gen date = time - 3650.5 + date("2050-01-01", "YMD")
. format date %td
.
. list in 1/10

```

	time	lat	lon	tas	date
1.	3650.5	24.125	108.125	288.02673	01jan2050
2.	3650.5	24.125	108.375	289.05478	01jan2050
3.	3650.5	24.125	108.625	288.97476	01jan2050
4.	3650.5	24.125	108.875	288.53751	01jan2050
5.	3650.5	24.125	109.125	289.27686	01jan2050
6.	3650.5	24.125	109.375	290.04214	01jan2050
7.	3650.5	24.125	109.625	289.90146	01jan2050
8.	3650.5	24.125	109.875	290.08304	01jan2050
9.	3650.5	24.125	110.125	286.86743	01jan2050
10.	3650.5	24.125	110.375	287.87402	01jan2050

```

.
. save "grid_all.dta", replace
file grid_all.dta saved

```

Calculate Average temperature for cities in Hunan

We then calculate the average temperature for Hunan Province using the `zonalstats` command. In this example, zonal statistics are applied to the temperature field on January 1, 2025. The resulting spatial distribution is shown in Figure 5, which depicts regional variation in daily mean temperature across Hunan.

```

. zonalstats tas_day_BCC-CSM2-MR_ssp245_r1i1p1f1_gn_2050.nc using "hunan.shp", ///
> var(tas) stats(avg) origin(1 `lat_start' `lon_start') ///

```

```

>                                     size(1 `lat_count` `lon_count`) crs(EPSG:4326) clear
NetCDF variable `tas` type: float
NetCDF variable `tas` missing value attribute: 1.OE20 (type: float)
NetCDF CRS not detected. Using user-provided CRS: EPSG:4326
Coordinate systems are compatible, no reprojection needed
Created numeric variable: avg
Data successfully exported to Stata dataset.

. replace avg = avg - 273.15
(14 real changes made)

. save "hunan_temp.dta", replace
file hunan_temp.dta saved

.

. use hunan.dta, clear

.

. rename Name z_Name

. merge 1:1 z_Name using hunan_temp.dta,nogen
(variable z_Name was str11, now str16 to accommodate using data's values)

      Result                                Number of obs
-----
Not matched                                0
Matched                                  14

. save hunan_temp.dta, replace
file hunan_temp.dta saved

.

. geoframe create region ///
>   "hunan_temp.dta", id(_ID) centroids(_CX _CY) ///
>   shp(hunan_shp.dta) replace
(creating frame region from hunan_temp.dta)
(creating frame region_shp from hunan_shp.dta)

      Frame name: region [make current]
      Frame type: attribute
      Feature type: <none>
      Number of obs: 14
      Unit ID: _ID
      Coordinates: _CX _CY
      Linked shape frame: region_shp

.

. geoplot ///
>   (area region avg, color(Greys) ///
>     level(6, quantile weight(avg))) ///
>   (line region, lwidth(vthin)), legend(position(sw))

```

Calculate 80km-radius IDW temperature for cities in Hunan

To move beyond area-based aggregation, we apply the `matchgeop` command to calculate city-level temperature using Inverse Distance Weighting (IDW) interpolation. Specifically, grid points within an 80 km radius of each city are incorporated into the calculation. This approach emphasizes localized climatic conditions at urban cores, providing a perspective better aligned with the lived environment of city residents than administrative averages alone.

```

. use "grid_all.dta", clear

```

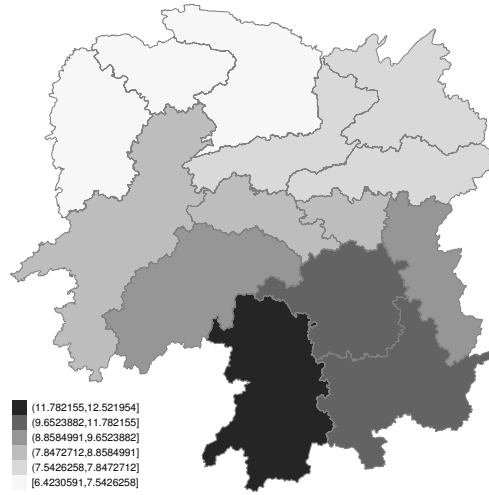



Figure 5: Average temperature in Hunan

```

. rename lon ulon
. rename lat ulat
. gen n=_n
. save "grid_all_2.dta", replace
file grid_all_2.dta saved

.
. use "hunan_city.dta", clear
. matchgeop ORIG_FID lat lon using grid_all_2.dta, ///
> neighbors(n ulat ulon) within(80) gen(distance)

.
. merge m:1 n using grid_all_2.dta, keep(3)
Result                                Number of obs
Not matched                             0
Matched                                148,555  (_merge==3)

. drop _merge
.
. list city ulat ulon distance date tas in 1/10

```

	city	ulat	ulon	distance	date	tas
1.	Yongzhou	25.125	111.625	72.995079	01jan2050	287.24945
2.	Yongzhou	25.125	111.875	73.701637	01jan2050	284.2619
3.	Chenzhou	25.125	113.125	76.69973	01jan2050	285.54437
4.	Yongzhou	25.375	111.125	75.168007	01jan2050	285.06604
5.	Yongzhou	25.375	111.375	56.90913	01jan2050	286.44476
6.	Yongzhou	25.375	111.625	45.657139	01jan2050	287.84082
7.	Yongzhou	25.375	111.875	46.776157	01jan2050	286.04608

8.	Yongzhou	25.375	112.125	59.573086	01jan2050	285.83316
9.	Yongzhou	25.375	112.375	78.532799	01jan2050	286.43384
10.	Chenzhou	25.375	112.625	70.880791	01jan2050	285.7345

```

.
. save "hunan_80km.dta", replace
file hunan_80km.dta saved
.
. drop if tas==.
(0 observations deleted)
. gen weight = 1/distance
. gen weighted_tas = tas * weight
. bysort city date : egen sum_weighted_tas = total(weighted_tas)
. bysort city date : egen total_weight = total(weight)
. gen idw_tas = sum_weighted_tas / total_weight
. gen temp_c = idw_tas - 273.15
.
. duplicates drop city date , force
Duplicates in terms of city date
(143,445 observations deleted)
.
. list city distance date temp_c in 1/10

```

	city	distance	date	temp_c
1.	Changde	59.276711	01jan2050	7.65365
2.	Changde	38.741573	02jan2050	7.220788
3.	Changde	38.741573	03jan2050	8.44021
4.	Changde	38.741573	04jan2050	6.136468
5.	Changde	39.613548	05jan2050	7.520319
6.	Changde	74.443283	06jan2050	7.391534
7.	Changde	39.140995	07jan2050	6.704889
8.	Changde	38.741573	08jan2050	3.407739
9.	Changde	60.863079	09jan2050	2.457117
10.	Changde	16.299124	10jan2050	2.555414

```

.
. save "hunan_IDW.dta", replace
file hunan_IDW.dta saved

```

Figure 6 presents the resulting IDW temperature distributions on January 1, 2050.

```

. use "hunan_IDW.dta" ,clear
. rename city Name
. merge m:1 Name using hunan.dta

```

Result	Number of obs	
Not matched	0	
Matched	5,110	(_merge==3)

```

.
. keep if date == date("01jan2050", "DMY")

```

```

(5,096 observations deleted)
. save hunan_IDW_202500101.dta, replace
file hunan_IDW_202500101.dta saved

.
. geoframe create region ///
> "hunan_IDW_202500101.dta", id(_ID) centroids(_CX _CY) ///
> shp(hunan_shp.dta) ///
> replace
(creating frame region from hunan_IDW_202500101.dta)
(creating frame region_shp from hunan_shp.dta)
      Frame name: region [make current]
      Frame type: attribute
      Feature type: <none>
      Number of obs: 14
      Unit ID: _ID
      Coordinates: _CX _CY
      Linked shape frame: region_shp

.
. geoplot ///
> (area region temp_c , color(Greys) ///
>   level(6, quantile weight(temp_c))) ///
> (line region, lwidth(vthin)), legend(position(sw))

```

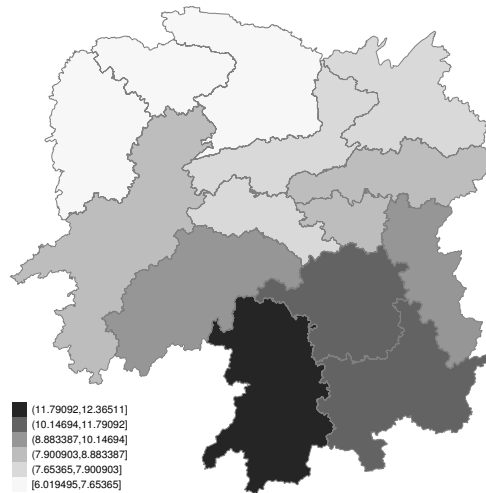


Figure 6: IDW Temperature at the geocentric center of cities in Hunan

9 Conclusions

Geospatial remote sensing data plays an indispensable role in contemporary social and economic research. To extend Stata's capabilities in handling such data, we develop a suite of commands that allow users to directly read and process GeoTIFF and NetCDF files within the statistical environment. This integration enables more seamless work-

flows and enhances reproducibility in empirical studies that rely on geospatial information.

While the commands provide substantial functionality, several limitations remain. First, implementation requires downloading the complete NetCDF and GeoTools libraries, occupying more than 100 MB of disk space. Second, many advanced GeoTools functions, such as buffer creation, raster cropping, and resampling, are not yet supported. Third, compared with dedicated GIS platforms such as ArcGIS, the current Stata framework lacks visualization features, making data operations less intuitive.

Despite these constraints, the commands significantly expand the scope of geospatial analysis in Stata, laying a foundation for future enhancements that may incorporate more advanced spatial operations and visualization capabilities.

10 Acknowledgments

Kerui Du thanks the financial support from the National Natural Science Foundation of China (Grant Nos. 72473119 and 72074184). Ruipeng Tan thanks the financial support from the National Natural Science Foundation of China (Grant No. 72403068) and Fundamental Research Funds for the Central Universities (JZ2025HGTB0235).

11 References

- Barwick, P. J., S. Li, L. Lin, and E. Y. Zou. 2024. From fog to smog: The value of pollution information. *American Economic Review* 114(5): 1338–1381.
- Burke, M., S. M. Hsiang, and E. Miguel. 2015. Global non-linear effect of temperature on economic production. *Nature* 527(7577): 235–239.
- He, G., T. Liu, and M. Zhou. 2020. Straw burning, PM_{2.5}, and death: Evidence from China. *Journal of Development Economics* 145: 102468.
- Henderson, J. V., A. Storeygard, and D. N. Weil. 2012. Measuring economic growth from outer space. *American Economic Review* 102(2): 994–1028.
- Jann, B. 2019. HEATPLOT: Stata module to create heat plots and hexagon plots. Statistical Software Components, Boston College Department of Economics. <https://ideas.repec.org/c/boc/bocode/s458598.html>.
- . 2023. GEOPLOT: Stata module to draw maps. Statistical Software Components, Boston College Department of Economics. <https://ideas.repec.org/c/boc/bocode/s459211.html>.
- Wu, K. C. Z., L. S. C. Z., Yizhen; Shi. 2021. An improved time-series DMSP-OLS-like data (1992-2024) in China by integrating DMSP-OLS and SNPP-VIIRS. <https://doi.org/10.7910/DVN/GIYGJU>.

About the authors

Kerui Du is an associate professor at the School of Management, Xiamen University, China. His primary research interests include applied econometrics, energy, and environmental economics.

Chunxia Chen is a master student in Technological Economics and Management at the School of Management, Xiamen University, China.

Yang Song is master student in Applied Economics at the School of Economics, Hefei University of Technology, China.

Shuo Hu is a Ph.D. student at the School of Economics, Southwestern University of Finance and Economics, China. His research interest is international environmental economics.

Ruipeng Tan (corresponding author) is a professor at the School of Economics, Hefei University of Technology, China. His primary research interests include energy and environmental economics, and innovation economics.